Algorytmy i struktury danych

Kolejki priorytetowe i kopce

Interaktywna aplikacja edukacyjna - dokumentacja

Nazwa tematu	Kolejki priorytetowe i kopce
Aplikacja	Kolejki priorytetowe i kopce - interaktywna wizualizacja
Autor	Łukasz Molis
Numer indeksu	91612
Przedmiot	Algorytmy i struktury danych
Rok akademicki	2026/2027
Data	26.10.2026

Technologia: HTML + CSS + JavaScript (zero zależności, praca lokalna)

1. Cel i zakres

1.1. Problem

Celem projektu jest zbudowanie kolejki priorytetowej (ang. priority queue) realizowanej na trzy różne sposoby oraz interaktywna wizualizacja, która krok po kroku pokazuje, jak działa każdy z algorytmów. Kolejka priorytetowa przechowuje zadania i zwraca je w kolejności malejącej ważności - zadanie o niższym priorytecie jest obsługiwane wcześniej.

1.2. Odbiorca

Dokument i aplikacja są przeznaczone dla studentów uczących się podstaw struktur danych i analizy algorytmów. Odbiorca powinien rozumieć pojęcie złożoności czasowej (notacja O) oraz podstawowe struktury tablicowe i drzewiaste.

1.3. Założenia

- Do 31 zadań; każde ma unikalny identyfikator (auto-inkrementacja) i priorytet 0-9.
- Mniejsza liczba = wcześniejsza obsługa; przy równym priorytecie wygrywa wcześniej dodane zadanie (FIFO).
- Wszystkie trzy warianty porównują tę samą parę: (priorytet, kolejność dodania).
- Operacje: dodaj, pokaż minimum, pobierz minimum; zmiana priorytetu nie jest wymagana.
- Program działa lokalnie - bez płatnego API i kont; gotowa biblioteka nie zastępuje badanego algorytmu.

1.4. Lista wykonanych wymagań

Wymaganie	Status
Tablica nieuporządkowana (dopisz + szukaj minimum)	Tak
Tablica uporządkowana (wstawianie + koszt porównań/przesunięć)	Tak
Kopiec binarny min (sift-up / sift-down)	Tak
Do 31 zadań, priorytet 0-9, reguła (priorytet, kolejność)	Tak
Operacje: dodaj / pokaż min / pobierz min	Tak
Interaktywny pokaz: Krok, Start/Pauza, Reset, zmiana tempa	Tak
3 gotowe scenariusze + własne dane użytkownika	Tak
Kroki wynikające z wykonania kodu (nie animacja nagrana)	Tak
Wyróżniony wiersz pseudokodu + przyczyna działania	Tak
3 pytania z odpowiedziami (nauka), legenda, opisy po polsku	Tak
Porównanie: liczniki, rozmiar wejścia, koszt czasowy i pamięciowy	Tak
10 automatycznych testów (w tym 2 stany pośrednie)	Tak
Oddzielny walidator, niezależny od algorytmów	Tak

2. Algorytmy i mechanizmy

2.1. Tablica nieuporządkowana

Zadania trzymane są w zwykłej tablicy w kolejności dodania. Dodanie nowego elementu to dopisanie na koniec (O(1), 0 porównań). Pobranie minimum wymaga przeskanowania całej tablicy (O(n), dokładnie n-1 porównań) i zapamiętania indeksu najmniejszego elementu.

```
function extractMin():
minIdx = 0
for i = 1 .. len-1:
if compare(a[i], a[minIdx]): minIdx = i
return remove(minIdx)
```

2.2. Tablica uporządkowana

Elementy utrzymywane są w porządku rosnącym według pary (priorytet, ID). Wstawianie znajduje pozycję przez porównania, a następnie przesuwa elementy w prawo (O(n) porównań + O(n) przesunięć). Minimum zawsze jest na pozycji 0, więc pobranie kosztuje O(1).

```
function add(item):
pos = 0
while pos for i = len .. pos+1: a[i] = a[i-1]
a[pos] = item
```

2.3. Kopiec binarny minimum

Kopiec przechowywany jest w tablicy indeksowanej jak drzewo (dzieci węzła i to 2i+1 i 2i+2). Wstawianie dopisuje element na koniec i przywraca porządek "rodzic <= dzieci" przez przesiewanie w górę (sift-up). Pobranie bierze korzeń, przenosi na jego miejsce ostatni element i przesiewa w dół (sift-down). Obie operacje kosztują O(log n).

```
function siftUp(idx):
while idx>0:
p = (idx-1)/2
if heap[idx] else: break

function siftDown(idx):
loop:
s = idx
if left if right if s==idx: break
swap; idx=s
```

2.4. Warunki stosowania

Tablica nieuporządkowana - gdy dodajemy znacznie częściej niż pobieramy. Tablica uporządkowana - gdy pobieramy częściej niż dodajemy i zbiór jest mały. Kopiec - gdy obie operacje są częste i zbiór duży (system czasu rzeczywistego, algorytm Dijkstry, kodowanie Huffmana).

3. Struktury danych i złożoność

3.1. Reprezentacja i stan

Wszystkie trzy warianty używają dynamicznej tablicy elementów. Element to para {id, priority}. Różnica polega na niezmienniku: tablica nieuporządkowana nie ma żadnego porządku; tablica uporządkowana utrzymuje sortowanie po (priority, id); kopiec utrzymuje własność kopca (rodzic <= dzieci) w tablicy indeksowanej drzewiasto.

3.2. Uzasadnienie struktur

Tablica daje lokalność pamięci i prostą implementację. Indeksowanie drzewiaste pozwala trzymać kopiec w tablicy bez wskaźników - dzieci i rodzica wylicza się arytmetycznie. To minimalizuje narzut pamięciowy i ułatwia wizualizację (poziomy drzewa to kolejne warstwy tablicy).

3.3. Złożoność czasowa i pamięciowa

Operacja	Tablica nieuporządkowana	Tablica uporządkowana	Kopiec binarny (min)
Wstawianie	O(1) / 0 porównań	O(n) porównań + O(n) przesunięć	O(log n) porównań + swapów
Pobranie minimum	O(n) - skan n-1 porównań	O(1) - element[0]	O(log n) - sift-down
Pamięć	O(n)	O(n)	O(n)
Struktura	-	-	log2(n+1) poziomów

4. Technologie

4.1. Język i biblioteki

Aplikacja napisana jest w czystym HTML + CSS + JavaScript (vanilla), bez żadnych bibliotek zewnętrznych ani frameworków. Całość mieści się w jednym pliku priority-queue-heaps.html.

4.2. Uzasadnienie wyboru

- Zero zależności - program działa od razu po otwarciu pliku, bez instalacji i budowania.
- Algorytmy są w kodzie projektu: funkcje-generatory (function*) zwracają kolejne kroki wykonania.
- Przeglądarka daje dostęp do DOM do wizualizacji kroków i animacji bez dodatkowego narzędzia.
- HTML/CSS/JS to technologia rekomendowana w karcie tematu.

4.3. Lokalne uruchomienie

Wystarczy dwukrotnie kliknąć plik priority-queue-heaps.html - otworzy się w domyślnej przeglądarce (Chrome, Edge, Firefox). Można też wystawić go przez GitHub Pages (włączyć Pages na gałęzi main). Nie jest wymagany serwer, internet ani konto.

5. Architektura i realizacja

5.1. Moduły (w obrębie jednego pliku)

- Warstwa wizualna (HTML + CSS) - zakładki, baner kroku, panele operacji/sterowania/statystyk.
- Definicje pseudokodu (obiekt pseudocode) - tekst trzech algorytmów z wyróżnieniem linii.
- Stan (obiekt states) - trzy struktury z licznikami porównań, przesunięć i swapów.
- Generatory kroków (unorderedScanGen, orderedAddGen, heapAddGen, heapExtractGen) - badane algorytmy.
- Silnik krokowania (enqueue → processQueue → executeStep) - kolejka kroków i odtwarzanie.
- Warstwa testująca (runTests) i walidator (klasa Validator) - niezależne od algorytmów.

5.2. Przepływ danych i powiązanie obliczeń z wizualizacją

Kliknięcie operacji tworzy generator. Generator wykonuje algorytm i na każdym kroku zwraca obiekt opisujący stan (typ kroku, indeksy, porównywane elementy, liczniki). Kroki trafiają do kolejki playQueue, a silnik odtwarza je po kolei - każdy krok wywołuje renderowanie tablicy/drzewa, wyróżnia wiersz pseudokodu i aktualizuje baner pokazujący bieżącą akcję. Dzięki temu wizualizacja jest ścisłym odwzorowaniem wykonania kodu, a nie nagraną animacją.

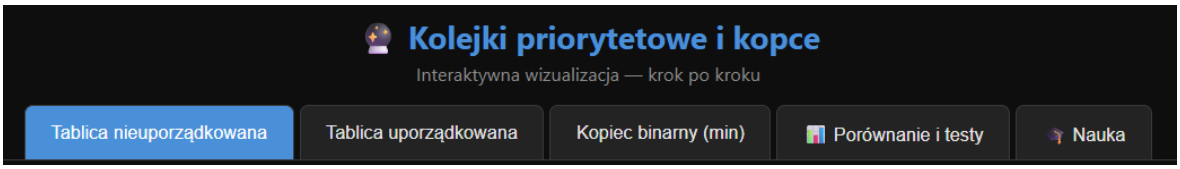
5.3. Etapy pracy

- 1) Implementacja trzech algorytmów jako generatorów kroków.
- 2) Silnik krokowania z pauzą / krokiem pojedynczym / tempem.
- 3) Wizualizacja tablic i drzewa kopca z legendą kolorów.
- 4) Scenariusze, nauka (pytania), testy i walidator.

6. Interfejs i obsługa

6.1. Widok ogólny i sterowanie

U góry znajduje się pasek zakładek wyboru struktury (trzy tablice oraz zakładki "Porównanie i testy", "Nauka"). Pod nim baner kroku pokazuje na bieżąco, jaką operację wykonuje algorytm. Po lewej jest wizualizacja i pseudokod, po prawej panel sterowania: operacje (Dodaj / Pokaż min / Pobierz min), scenariusz, sterowanie krokami (Krok / Start / Reset / tempo) oraz statystyki i stan.



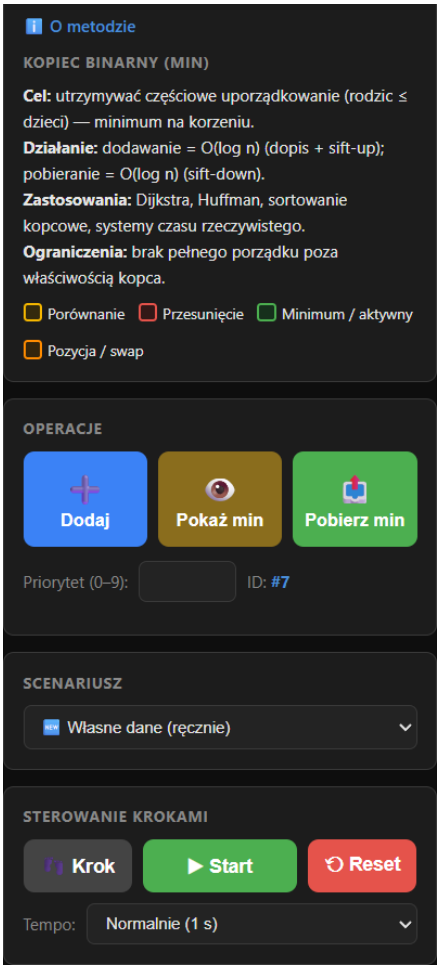
Rysunek 1. Widok główny aplikacji - zakładka "Tablica nieuporządkowana"



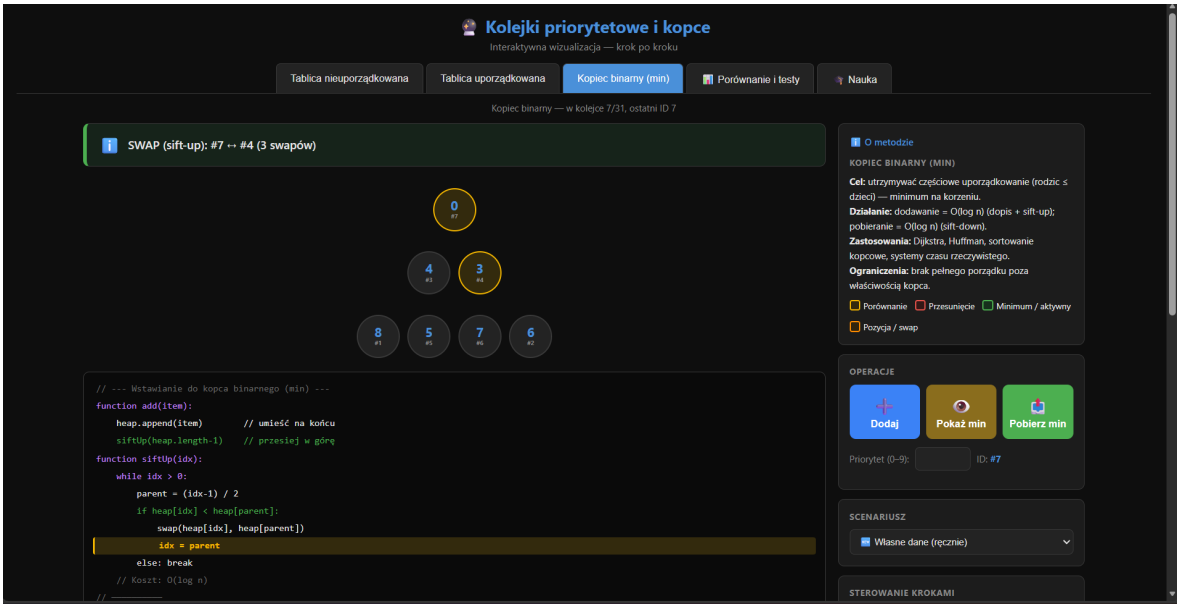
Rysunek 2. Pseudokod z wyróżnioną linią

6.2. Zmiana danych

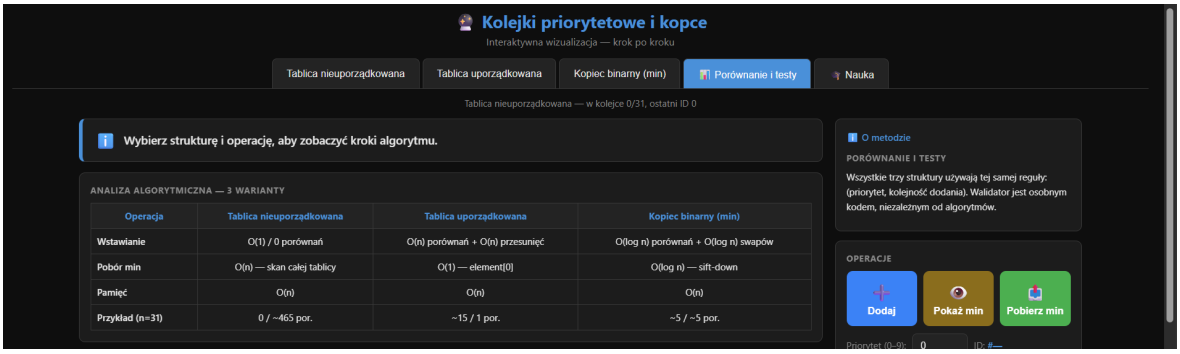
Priorytet wprowadza się w polu obok przycisku Dodaj. Scenariusz zmienia się listą rozwijaną (3 gotowe zestawy + dane własne). Każda zmiana scenariusza zeruje stan i zaczyna nowy przebieg.



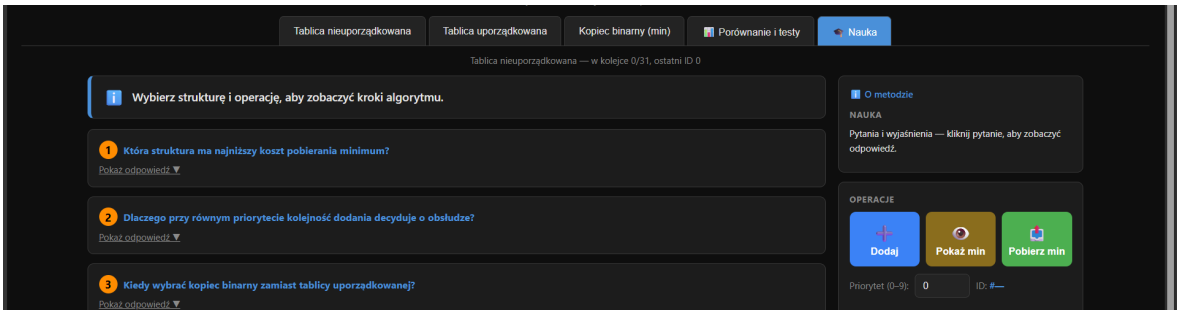
Rysunek 3. Panel sterowania - operacje, scenariusz, sterowanie krokami



Rysunek 4. Wizualizacja kopca binarnego



Rysunek 5. Zakładka "Porównanie i testy" - tabela złożoności i 10 testów



Rysunek 6. Zakładka "Nauka" - pytania i odpowiedzi

7. Porównanie metod

7.1. Dane i liczniki

Porównanie wykonano na n = 31 zadań (pełny zestaw z priorytetami 0-9). Liczniki mierzą rzeczywiste operacje: porównania (wszystkie warianty), przesunięcia (tablica uporządkowana) i swapy (kopiec).

7.2. Rzeczywiste wyniki

Wskaźnik	Nieuporządkowana	Uporządkowana	Kopiec
Porównań - dodanie 1 el.	0	ok. n/2 = 15	do 5 (log2 31)
Porównań - pobranie 1 el.	n-1 = 30	1	do 5
Suma porównań (31 pobrań)	465 = n(n-1)/2	31	ok. 150-160
Przesunięcia / swapy	0	ok. n/2 na dodanie	ok. log n na operację

7.3. Interpretacja

Tablica nieuporządkowana jest najtańsza przy dodawaniu (0 pracy), ale najdroższa przy pobieraniu (każde pobranie to pełny skan - dla 31 elementów łącznie 465 porównań). Tablica uporządkowana odwraca ten koszt: pobranie jest natychmiastowe, lecz każde dodanie wymaga średnio ok. n/2 porównań i ok. n/2 przesunięć. Kopiec bilansuje obie operacje na poziomie logarytmicznym, co dla dużego n daje najlepsze zachowanie przy mieszanym obciążeniu.

8. Testowanie

8.1. Uruchomienie testów

Testy uruchamiają się automatycznie przy starcie aplikacji (zakładka "Porównanie i testy"). Każdy test ma zdefiniowane wejście, oczekiwany wynik i status (PASS/FAIL). Poniżej podsumowanie 10 testów.

#	Test	Wejście	Wynik oczekiwany	Uzyskany
1	Minimum z tablicy nieuporządkowanej	31 par	id = prawdziwe minimum	PASS
2	Utrzymanie porządku (uporządkowana)	31 par	sort wg (priorytet, id)	PASS
3	Własność kopca po 31 wstawieniach	31 par	rodzic <= dzieci	PASS
4	Ten sam porządek ekstrakcji (3 struktury)	te same 31 par	3 identyczne sekwencje	PASS
5	FIFO dla równego priorytetu (kopiec)	5 zadań p=3	kolejność 1,2,3,4,5	PASS
6	Stan pośredni: korzeń kopca po 3 wstawieniach	#1/p5,#2/p3,#3/p7	korzeń = #2 (p=3)	PASS
7	Stan pośredni: uporządkowana po 3 wstawieniach	#1/p9,#2/p1,#3/p5	[#2,#3,#1]	PASS
8	Ekstrakcja z kopca + zachowana własność	7 elementów	min=#4/p1	PASS
9	Integralność: 31 zadań, poprawne ID	seed=12345	31 par, ID 1-31	PASS
10	Zakres priorytetów 0-9	31 par	wszystkie w [0,9]	PASS

8.2. Przypadki brzegowe

- Struktura pusta - operacje pobrania są blokowane i nie powodują błędu.
- Jeden element - pobranie z kopca zwraca go bez sift-down.
- Wszystkie równe priorytety - zachowana kolejność FIFO (test 5).
- Maksymalny rozmiar (31) - nadmiarowe dodanie jest odrzucane.

8.3. Poprawione błędy

- Niewłaściwy zapis div.dataset-line (kolizja z właściwością DOM) - zamienione na dataset.lineNum.
- Niezamknięty literał szablonu w wierszu wyników testów.
- Brak definicji funkcji compareHeap - uzupełniona przy stanie globalnym.
- Niespójne nazwy pól p / priority w scenariuszach i testach - ujednolicone na priority.
- Nadmiarowy warunek w funkcji loadScenario - usunięty.
- Błędny klucz wyróżniania pseudokodu dla kroków swap (sift-up vs sift-down) - dodane rozróżnienie fazy.

9. Podsumowanie i źródła

9.1. Wnioski

Trzy implementacje pokazują klasyczny kompromis między kosztem dodawania i pobierania. Kopiec binarny jest najlepszym rozwiązaniem ogólnego przeznaczenia (O(log n) na operację), podczas gdy tablice sprawdzają się w skrajnych profilach użycia. Interaktywna wizualizacja kroków skutecznie tłumaczy działanie algorytmów, zwłaszcza przesiewania (sift-up/sift-down), które są trudne do zrozumienia wyłącznie z opisu tekstowego.

9.2. Ograniczenia

- Maksimum 31 zadań i priorytet 0-9 - narzucone przez kartę tematu.
- Brak operacji zmiany priorytetu (change-key) - poza zakresem karty.
- Wizualizacja kopca to uproszczone drzewo bez jawnych krawędzi (poziomy jako warstwy).

9.3. Możliwości rozwoju

- Zmiana priorytetu z aktualizacją pozycji (decrease-key dla Dijkstry).
- Kopiec z jawnymi krawędziami i animacją zamiany węzłów.
- Sortowanie kopcowe (heapsort) jako dodatkowy scenariusz.
- Eksport historii kroków i generowanie losowych danych o zadanym rozkładzie.

9.4. Materiały i narzędzia

- Cormen T. H., Leiserson C. E., Rivest R. L., Stein C. - "Wprowadzenie do algorytmów" (rozdział o kopcach i kolejkach priorytetowych).
- Sedgewick R., Wayne K. - "Algorithms" (priority queues, binary heap).
- Dokumentacja MDN: HTML, CSS, JavaScript (funkcje generatorów function*).

9.5. Narzędzia AI i kontrola wyników

Do przygotowania kodu i dokumentacji użyto asystenta AI. Wszystkie wyniki zostały zweryfikowane ręcznie: testy jednostkowe (10/10 PASS) uruchomione w Node.js, składnia JavaScript sprawdzona poleceniem node --check, a wyniki algorytmów porównane między trzema niezależnymi implementacjami (wszystkie dają identyczną kolejność pobierania). AI nie zastępuje badanego algorytmu - algorytmy są napisane jawnie w kodzie projektu.